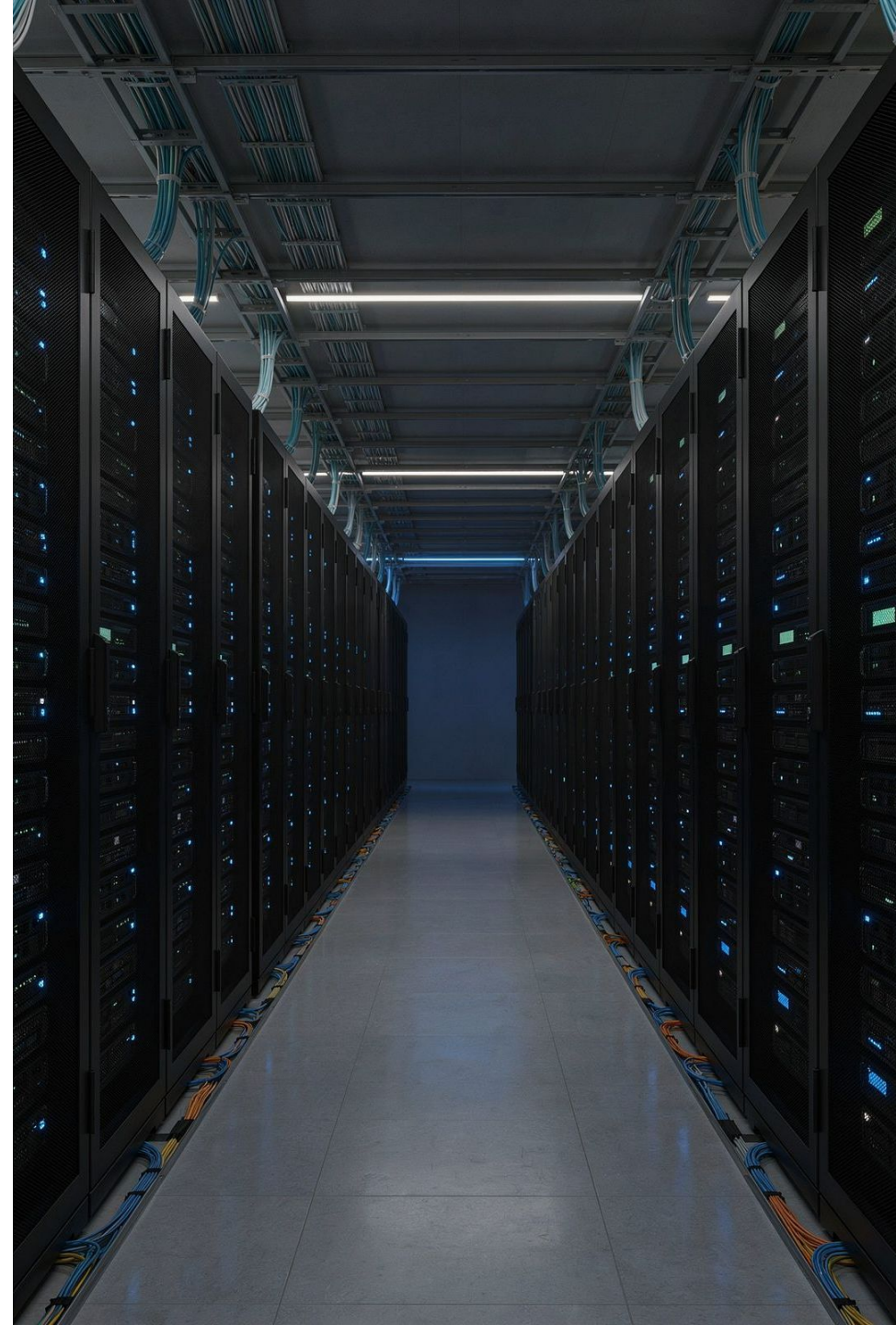


DevSecOps Introduction and Secure Engineering Flow

SESSION 6

Building Security into Every Stage of Software Delivery



Session Focus

Principles

DevSecOps & Secure SDLC

Controls

CI/CD gates, SAST, DAST, SCA

Secrets

Dependency & secrets security

Governance

Enterprise auditability & mortgage security

- ❏ Security is not a final checkpoint — it is continuously integrated into every stage.



Learning Objectives

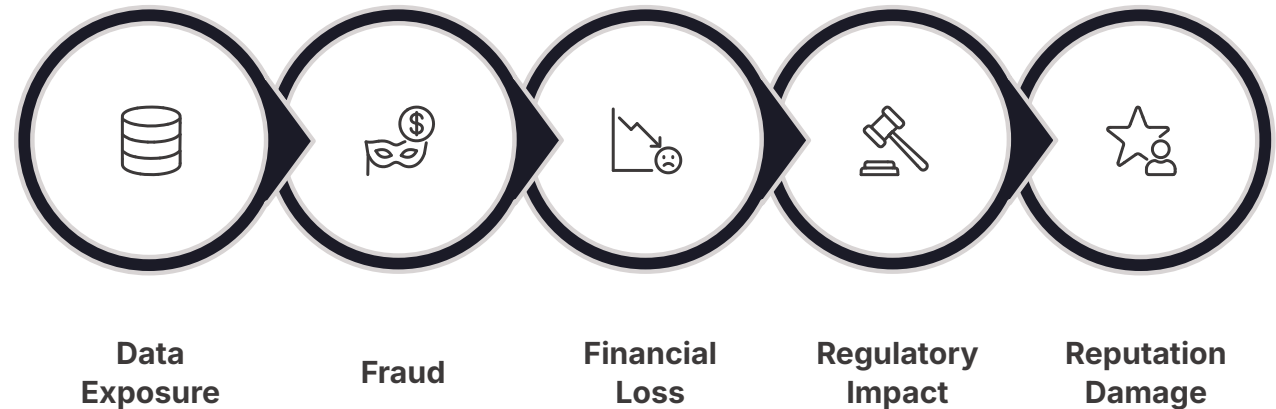
- DevOps vs DevSecOps
- Secure SDLC & shift-left
- SAST, DAST & SCA
- Secret & dependency scanning
- Security quality gates
- Secure PR practices
- Vulnerability remediation
- Enterprise evidence & mortgage data protection

Why Application Security Matters



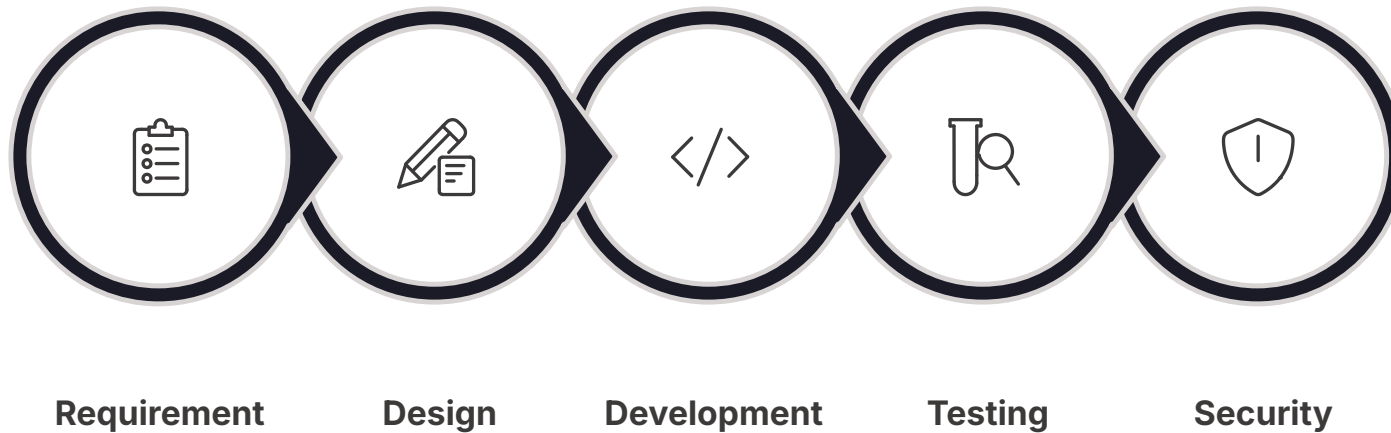
Mortgage Data at Risk

- Name, DOB, government IDs
- Income, bank & credit history
- Property info & payment history



Security is everyone's responsibility — developers, testers, architects, DevOps, and ops teams all contribute.

Traditional Security: Too Late



The Problem

After **3 months** of development, a final scan reveals:

Result: delays, expensive fixes, emergency remediation, and technical debt.

8 Critical

12 High

15 Medium

What Is DevOps?

Development + Operations — delivering software **faster, more frequently, and more reliably** through automation.



CI/CD pipelines automate and connect every stage.

What Is DevSecOps?

Old Way

Build App

**Security Team
Tests**

DevSecOps Way

Design Securely

Code Securely

Auto Scan

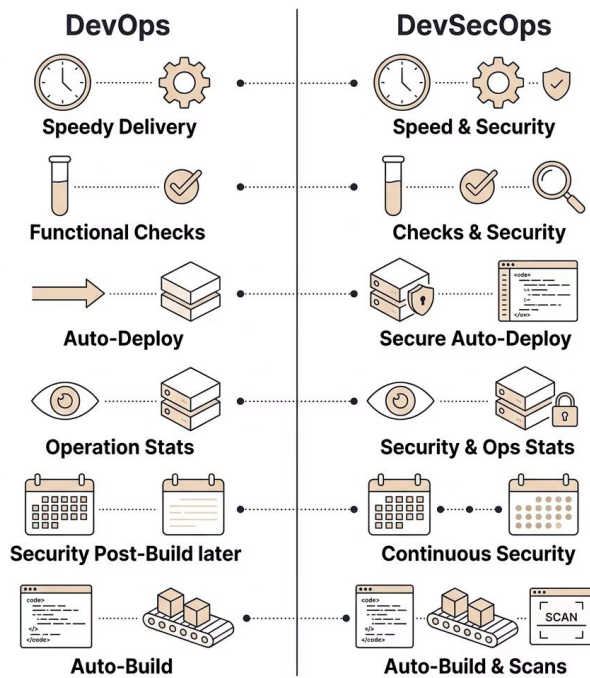
Test Security

Secure Deploy

Monitor

❏ Security becomes part of engineering — not a final checkpoint.

DevOps vs DevSecOps



Pipeline Comparison

DevOps: Commit → Build → Test → Deploy

DevSecOps:

- 1 Commit
- 2 Secret Scan
- 3 SAST
- 4 SCA
- 5 Container Scan
- 6 DAST → Deploy

Three Foundations of DevSecOps



People

Security is a shared responsibility.
Developers must understand
secure coding.



Process

Security in Definition of Done,
PRs, release process &
architecture reviews.

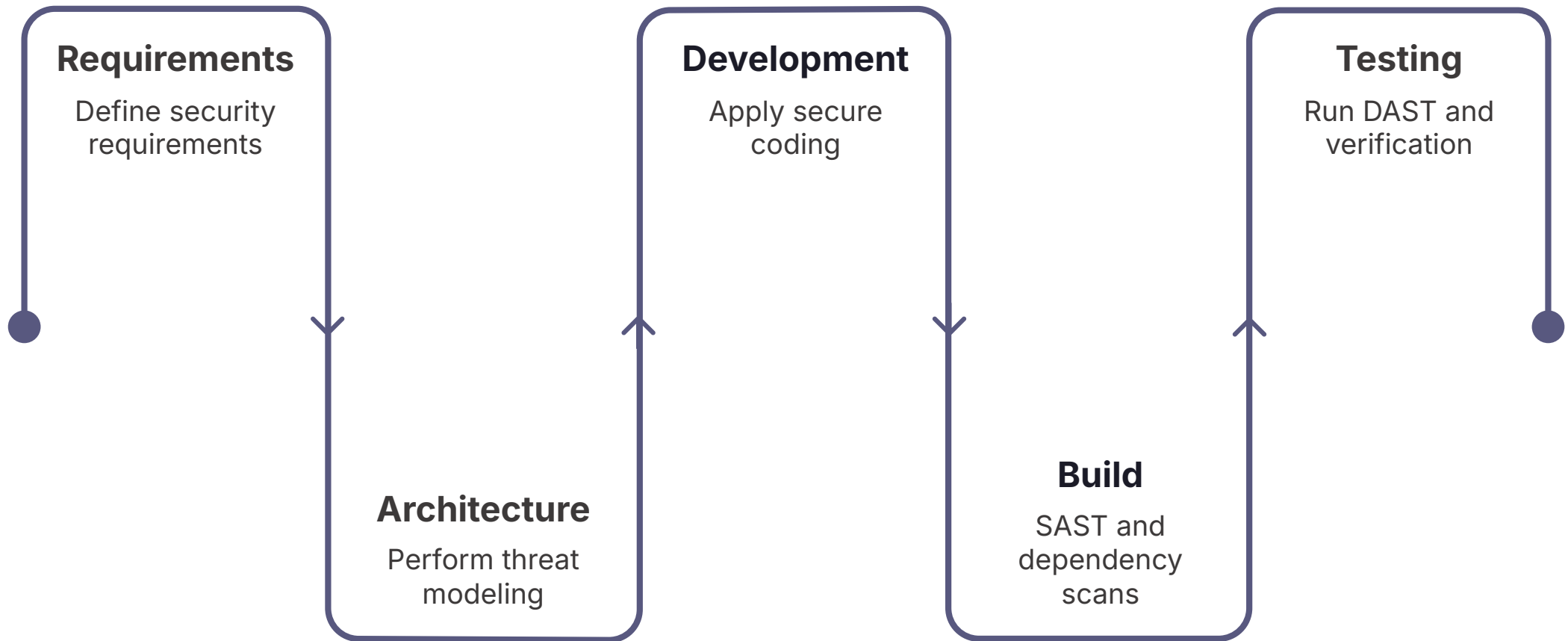


Technology

SAST, SCA, secret scanning,
DAST, container scanning & policy
enforcement.

People + Process + Automation = Effective DevSecOps

Secure Software Development Lifecycle



Every stage of the SDLC receives dedicated security controls — security is not an afterthought.

Security Across the SDLC

	Requirements Auth, encryption, audit logging, data protection
	Design Threat modeling, trust boundaries, secure architecture
	Development Secure coding, input validation, secret management
	Testing SAST, SCA, DAST, security tests
	Production Monitoring, vulnerability management, incident response

What Is Shift-Left Security?



Traditional: Security at the Right

Requirements → Code → Build → Test
→ Deploy → **Security**

Shift-Left: Security at the Start

Security → Requirements → Design →
Code → Build → Deploy

- ❏ Find security problems as early as possible.

Why Shift Left? The Cost of Late Detection

Scenario A — Caught During Coding

```
dbPassword = "Mortgage@123"
```

Secret scanner flags it immediately.

Fix time: **5 minutes**

Scenario B — Found After Production

- Remove secret from code
- Rotate password
- Review logs & repo history
- Investigate unauthorized access
- Redeploy services

📄 The earlier we find it, the cheaper and safer it is to fix.

Secure Developer Workflow



Security begins **before** code reaches the main branch.

CI/CD Security Pipeline



Each stage answers a different security question before software advances.

Security Quality Gates



"Is this software safe enough to proceed?"

Critical Vulns > 0 → **BLOCK**

High Vulns > Threshold → **BLOCK**

Secrets Found → **BLOCK**

Unit Tests Failed → **BLOCK**

Only compliant builds continue to the next stage.

Security Gate in Action

Build 	Unit Tests 	SAST 
Dependency Scan 	Secret Scan  AWS Credential Detected	

 **RELEASE BLOCKED** — Functional correctness does not automatically mean security correctness.

SAST: Static Application Security Testing

What It Does

Analyzes source code for security weaknesses —
"security analysis from inside the code."

When

Coding → Pull Request → Build

Typical Findings

- **SQL injection risks**
- **Hardcoded credentials**
- **Unsafe APIs & weak cryptography**
- **Improper input handling**
- **Path traversal & misconfigurations**

SAST Example: SQL Injection

Vulnerable Code

```
SELECT * FROM borrower  
WHERE name = "" + userInput + "";
```

SAST detects the unsafe string concatenation pattern.

Secure Fix

Use a **parameterized query** — the vulnerability is discovered **before deployment**.

- ✔ SAST catches the risk at code review, not in production.

DAST: Dynamic Application Security Testing



Tests a Running Application

DAST sends real requests against the web app, API, or running service to identify externally observable weaknesses.

SAST: "Does the code contain dangerous patterns?"

DAST: "Can the running app actually be attacked?"

DAST Example: Broken Authorization

Normal Request

```
GET /api/loans/10001
```

Manipulated Request

```
GET /api/loans/10002
```

The Question

Can **Borrower A** access **Borrower B's** mortgage information?

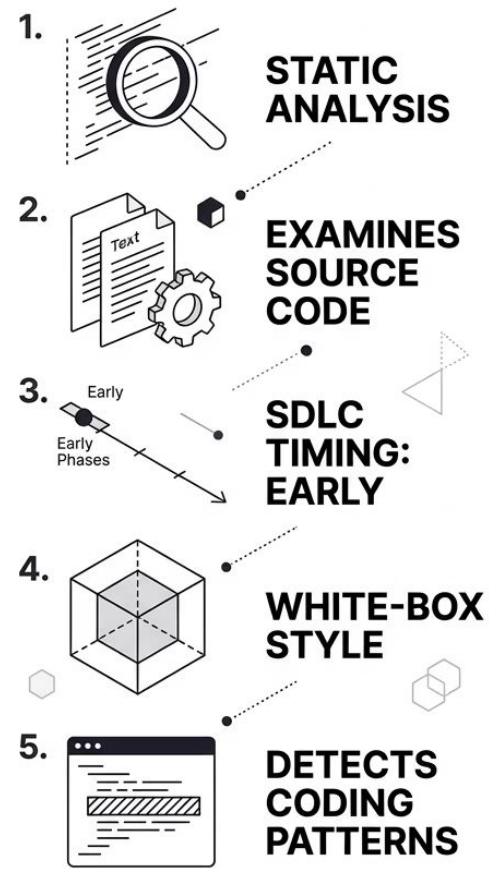
If yes → broken access control vulnerability.



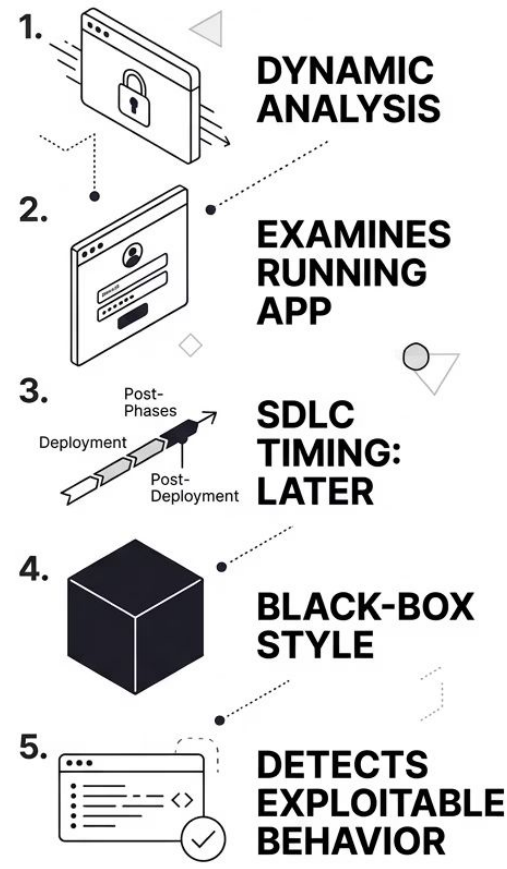
DAST tests application behavior from an attacker's perspective.

SAST vs DAST

SAST



DAST



Best Practice

✗ SAST OR DAST

✓ SAST + DAST

They solve different problems. Use both.

What Is SCA?

Modern Apps Rely on Dependencies

Spring Boot

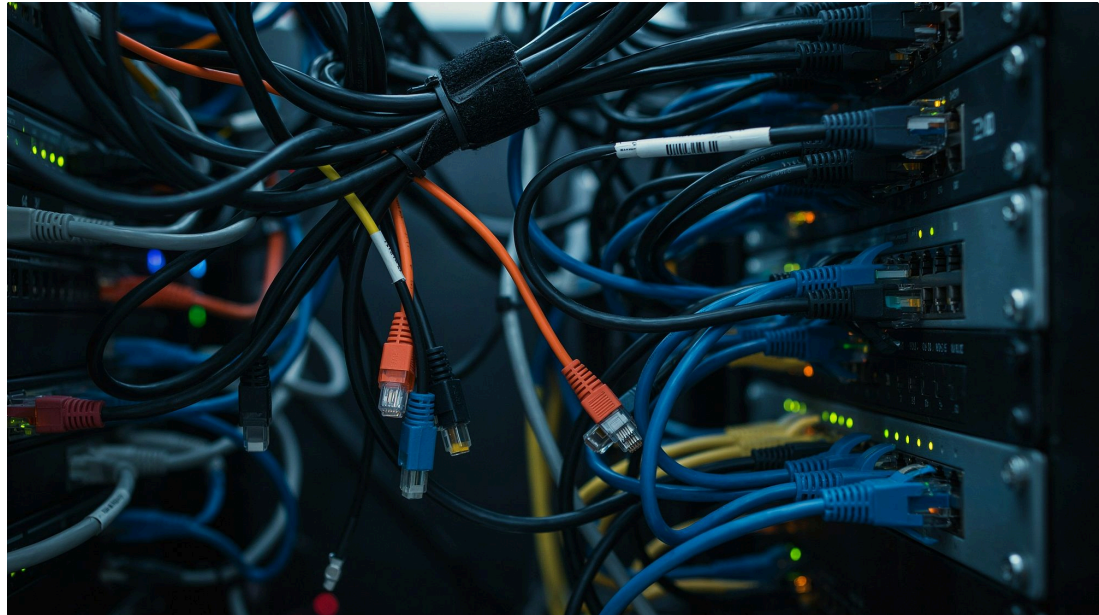
Jackson

Logging Libraries

DB Driver

JWT Library

AWS SDK



❏ **Software Composition Analysis (SCA)** — even if your code is secure, a dependency could contain a known vulnerability.

Dependency Security with SCA



Example finding: **Library xyz v2.1 — Severity: Critical — Fixed in v2.4**
Developer upgrades the dependency before release.

Secret Scanning

Common Mistakes Committed to Git

- Database passwords
- AWS access & secret keys
- API keys & OAuth secrets
- Private keys & certificates

What Secret Scanners Inspect

Source Code

Configuration

Commit History

Pull Requests

Scans for credential-like patterns across all of the above.

What If a Secret Is Committed?

Deleting the line is NOT enough

Git maintains history:

```
Commit 1 → Secret Added  
Commit 2 → Secret Removed
```

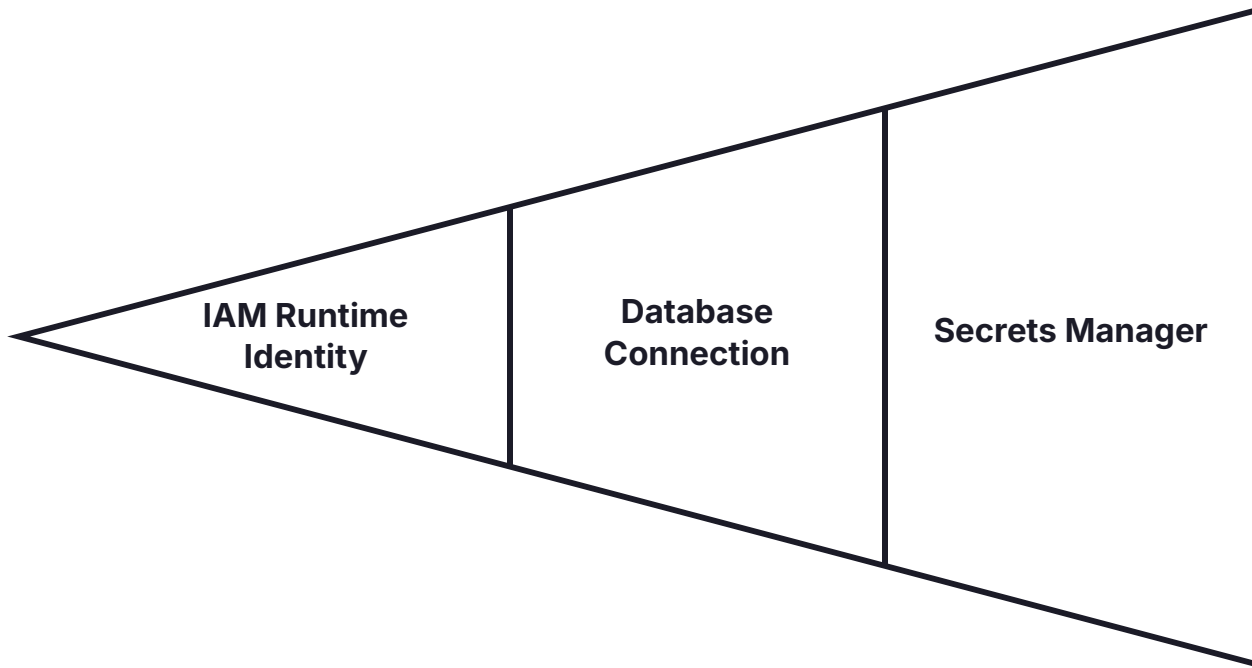
The secret may still exist in repository history.

Incident Response Steps

1. Treat credential as compromised
2. Revoke and rotate immediately
3. Remove exposure from repo
4. Review repository history
5. Investigate possible usage
6. Document incident if required

⊗ Never trust a leaked secret again. Rotate it.

Secure Secrets Management



Instead of hardcoded passwords in config files, use a managed secret store.



Centralized & Audited

All credentials in one governed location with full audit trail.



Rotation & Separation

Automatic rotation and environment separation reduce exposure.

Pull Request Security

A PR should ask more than "Does this code work?"

Input validated? Secrets hardcoded?

Authorization enforced? SQL parameterized?

Errors handled securely? Dependencies trusted?

Could this expose borrower data?



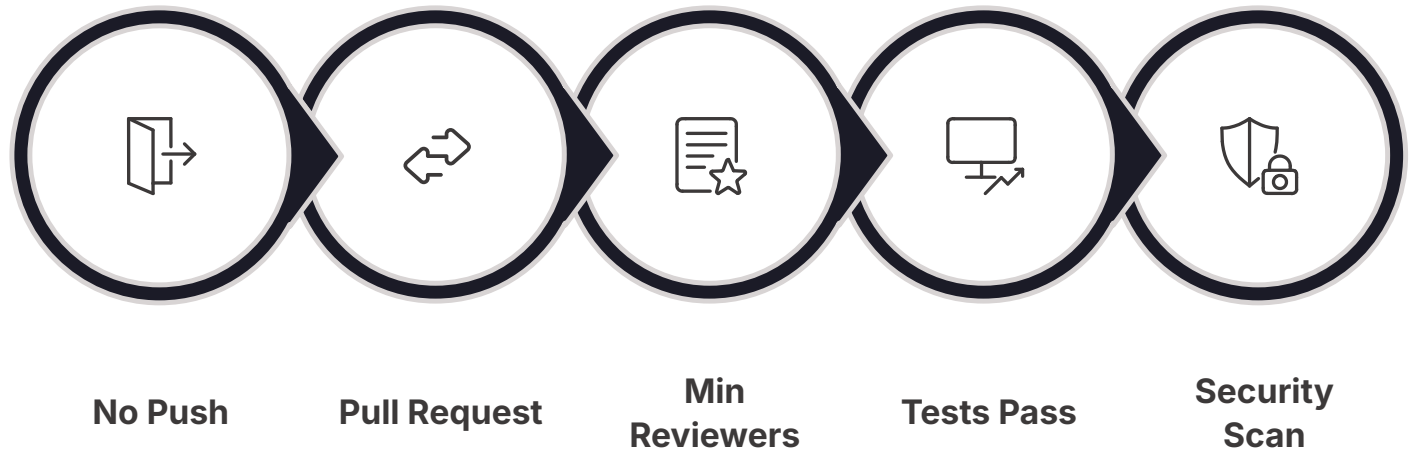
Branch Protection

Protected Branches

main

release

production



Reduces the chance that unreviewed or vulnerable code reaches production.

Vulnerability Severity & Remediation

1	Low Limited impact, minimal risk
2	Medium Weakness requiring specific conditions
3	High Significant auth/authorization weakness
4	Critical Remote compromise or severe borrower-data exposure

Organizations establish remediation SLAs based on risk and severity.

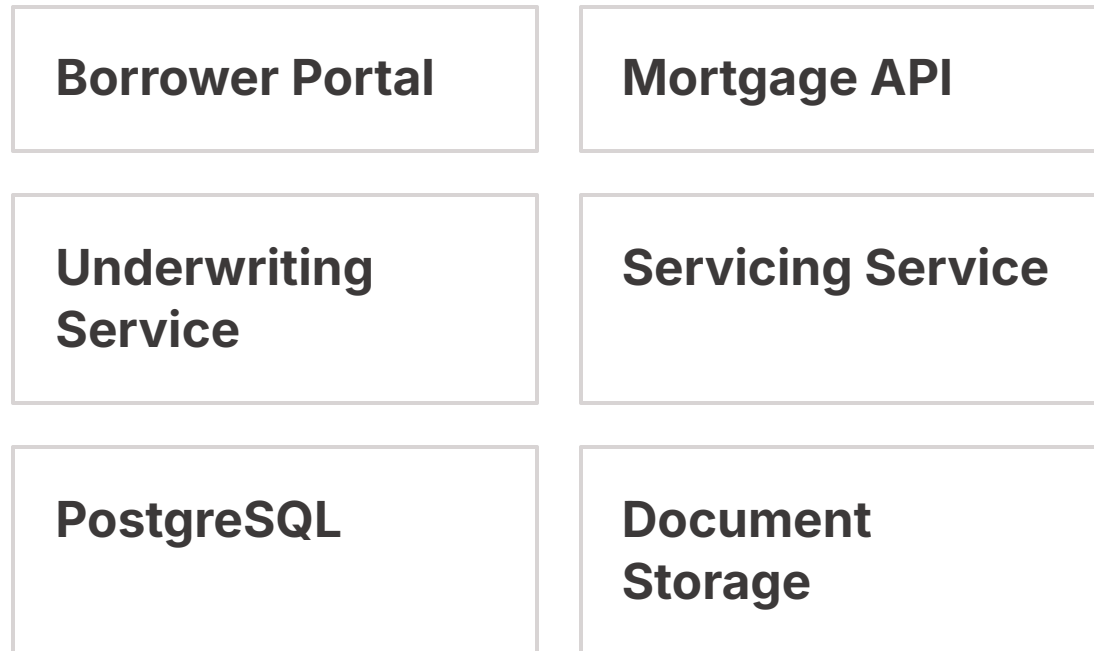
Enterprise Auditability & Evidence



"How do you know v2.5 was security tested before production?"

- **PR approvals & pipeline history**
- **SAST, SCA & DAST reports**
- **Secret scan & test results**
- **Deployment approvals & release records**
- **Vulnerability remediation records**

Mortgage Platform: Risks at Every Layer

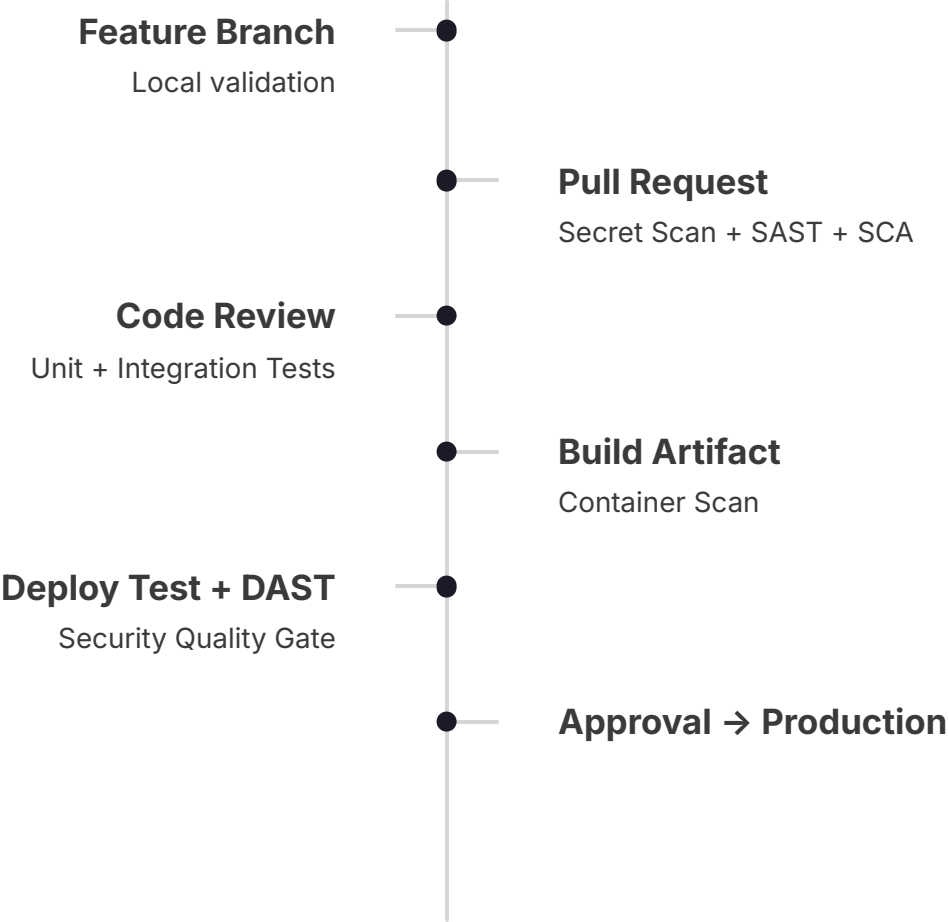


Potential Risks

- Borrower PII exposure
- Unauthorized loan access
- SQL injection & leaked credentials
- Vulnerable libraries & insecure APIs
- Sensitive data in logs
- Unauthorized production deployments

DevSecOps establishes automated controls around every risk.

Secure Mortgage Delivery Pipeline



Security becomes continuous, automated, and auditable.

Session Summary: Key Takeaways

1 Security is everyone's responsibility

DevSecOps integrates dev, security & ops.

2 Shift security left

Finding vulnerabilities early is far cheaper than post-deployment.

3 Automate security

SAST + SCA + Secret Scanning + DAST throughout CI/CD.

4 Enforce quality gates

Critical failures must stop vulnerable software from progressing.

5 Protect mortgage data & maintain evidence

Borrower PII requires strong controls — and proof they were executed.